Presentation Transcript

Multi-Modal Deep Learning for Fashion Product Classification

Team: Yiğit Dağıdır · İlayda Baburoğlu · Sevil Begüm Gürcan **Estimated run-time:** about 13 minutes (target 10–15) **Who speaks when:** Yiğit — slides 1–5 · İlayda — slides 6–9 · Sevil — slides 10–14

Each section below lines up with one slide of the deck. The timing in brackets is approximate; read at a calm, normal pace and the whole talk lands around 13 minutes.

Slide 1 — Title · Yiğit · (~38s)

Yiğit: Good morning everyone, and thank you for being here. We are Yiğit, İlayda and Sevil, and this is our Deep Learning project: *Multi-Modal Deep Learning for Fashion Product Classification*.

The idea in one sentence: every product in an online store has two things — a photo and a short text description. We built a model that looks at *both* together to decide what kind of product it is. And the real question behind the project is whether using both actually beats using just the picture, or just the text, on its own. Let me set that question up properly.

Slide 2 — The question · Yiğit · (~42s)

Yiğit: Here is the central question of the whole project: *does combining image and text beat using either one alone?*

The input is a product image plus its text description — a field called `productDisplayName`, something like “Nike Men Black Casual Shoes”. The output is the product’s category — the dataset calls it `subCategory` — narrowed to the ten most common categories.

To answer the question fairly, we don’t build one model — we build **three**, all on the same pipeline: an image-only model, a text-only model, and the fusion model that uses both. Comparing those three is the whole point of the project; everything else exists to make that comparison honest.

Slide 3 — Dataset · Yiğit · (~62s)

Yiğit: For the data we used the Kaggle *Fashion Product Images* dataset — the small version, light enough for a free Colab GPU. It’s a good fit because it ships all three things we need in one place: the images, the category labels, and a text field.

One key decision was *which* label to predict. We picked `subCategory`, and only the top-10 most frequent classes. Why? The very broad `masterCategory` is almost perfectly solved by the picture alone, so fusion would add nothing visible. At the other extreme, `articleType` has over 140 classes — too hard to train fairly in our timeline. The top-10 `subCategory` is the sweet spot, where the second modality can make a measurable difference.

Finally, we split the data 70% train, 15% validation, 15% test — a stratified split with a fixed seed, so every class keeps its proportion and all three models see the exact same split.

Slide 4 — Architecture overview · Yiğit · (~56s)

Yiğit: Here's the whole architecture on one slide — follow the two paths.

The **image** goes into EfficientNet-B0, a convolutional network, then a pooling step, then a small dense layer that produces 256 numbers summarising the picture.

The **text** goes through a vectorizer, an embedding layer, a GRU — a recurrent network — and a dense layer that produces 128 numbers summarising the description.

For the fusion model, we glue those two summaries together, pass them through one more dense layer with dropout, and finish with a softmax that gives a probability for each of the ten classes.

The key detail for fairness: the image-only and text-only baselines reuse the *identical* encoders, so any difference in the results comes from the model design, not from different data handling. It's all built with the Keras functional API. Now let me zoom into the image side.

Slide 5 — Image branch (CNN) · Yiğit · (~80s)

Yiğit: The image branch is a convolutional neural network — three core ideas.

First, **convolution**. Instead of connecting every pixel to every neuron, we slide small filters across the image. Each filter looks for a local pattern — an edge, a texture, a curve — and because the same filter is reused everywhere, the network spots that pattern wherever it appears. That property is called translation equivariance, and it's exactly the right assumption for images.

Second, the **activation** — a nonlinearity after each convolution. The classic choice is ReLU, which keeps positive values and zeros out the rest. EfficientNet uses a smoother version called Swish, but the idea is the same: it lets the network learn non-linear shapes.

Third, **pooling**. At the end, global average pooling averages each feature channel down to a single number — no parameters, just a compact summary.

For the backbone we use EfficientNet-B0, pre-trained on ImageNet, and we **freeze** it — reusing those general visual features as-is. On a modest dataset that avoids overfitting, trains much faster, and stays

stable. One honest caveat: the images are only about 60 by 80 pixels, upscaled to 224, so this branch works with limited detail.

That gives our 256-number picture summary. İlayda will now take the text side.

Slide 6 — Text branch (Embedding + GRU) · İlayda · (~80s)

İlayda: Thanks Yiğit. A product description is just a short string of words, and we need to turn it into numbers.

First we tokenise: lowercase, strip punctuation, and map each word to an integer. Each integer is then looked up in an **embedding** table that turns every word into a vector. This table is *trainable* — learned along with the model — so words with similar roles, like “shirt” and “tee”, end up with similar vectors. We deliberately avoided GloVe or BERT: the descriptions are short and domain-specific, so a small embedding we train ourselves is lighter and works just fine.

Then comes the **GRU**, our recurrent network. It reads the description word by word and keeps a running memory — the hidden state. What makes it smart is two gates. The **update gate** decides how much of the old memory to keep versus how much new information to write in. The **reset gate** decides how much of the past to forget when forming the new memory.

That gating matters for a concrete reason: it gives gradients a path to flow across the whole sequence instead of shrinking to zero — the classic vanishing-gradient problem with plain recurrent networks. We take the GRU’s final memory, pass it through a dense layer, and get our 128-number text summary.

Slide 7 — Fusion · İlayda · (~66s)

İlayda: Now the interesting part — putting the two together.

Fusion is simple to state: we take the 256 image numbers and the 128 text numbers and **concatenate** them into one vector of 384, then pass that through a dense layer with ReLU.

Why does that help, mathematically? Because each neuron sees *both* modalities at once. So a neuron can fire only when a visual feature **and** a textual feature appear together — say a “lace-up texture” in the image *and* a “shoe” word in the text. A simple average of two separate classifiers can’t represent that joint condition; this layer can. We call those cross-modal conjunctions.

The practical payoff is tie-breaking: when the image is ambiguous — a low-resolution sandal that looks like a shoe — the text settles it, and the other way around too. And because this fusion head is the only large, freely-trainable part of the model, we put a dropout of 0.3 on it to prevent overfitting.

Slide 8 — Loss & optimisation · İlayda · (~68s)

İlayda: All three models end the same way, so let me explain how they learn.

The final layer is a **softmax**, which turns the model's raw scores into probabilities that add up to one across the ten classes. We measure error with **cross-entropy** loss, which punishes the model for putting low probability on the correct class. A neat fact makes this stable to train: the gradient of softmax-plus-cross-entropy is just "predicted minus true" — a clean, bounded signal.

To minimise that loss we use **Adam**. Instead of one fixed learning rate, Adam keeps a separate, adaptive step size for every parameter — combining momentum, a smoothed average of recent gradients, with a sense of how large those gradients have been. That's especially useful here, because one model mixes a frozen CNN, a trainable embedding, and a recurrent network, all with very different gradient scales.

We also use early stopping — we watch the validation loss and stop once it stops improving for three epochs, restoring the best weights — and we fix the random seed to 42 everywhere for reproducibility.

Slide 9 — Experimental setup · İlayda · (~50s)

İlayda: Quickly, the concrete settings, so the results are reproducible. Adam with a learning rate of 0.001, sparse categorical cross-entropy loss, batch size 32, up to 15 epochs with early stopping. Images are 224 by 224 into the frozen EfficientNet-B0. On the text side: a vocabulary of 10,000 words, sequences of length 20, embeddings of size 128, and a GRU with 128 units.

For evaluation we use the held-out stratified test set rather than k-fold cross-validation — a deliberate trade-off, since k-fold would multiply our compute for precision we don't really need. Every one of these numbers lives in a single file, `src/config.py`, so there are no magic numbers scattered around. Sevil will now walk you through what we found.

Slide 10 — Results: the three-way comparison · Sevil · (~62s)

Sevil: Thank you İlayda. Here are the results on the held-out test set — 3,000 products the models never saw during training.

The image-only model reached about 98.3% accuracy, with a macro-F1 of 0.9727. The text-only model was the strongest, at 99.7% accuracy and a macro-F1 of 0.9949. And fusion came in at 99.5% accuracy and a macro-F1 of 0.9905.

A quick note on why we report **macro-F1** and not just accuracy: our ten classes are imbalanced, and macro-F1 weights every class equally instead of letting the big classes dominate.

Now the honest headline. The text channel basically **saturates** this task — product names very often state the category almost word-for-word, so text alone is already near-perfect. That leaves fusion almost no room to beat it: fusion effectively **ties** the text model — the gap is about four thousandths of a point, well inside run-to-run noise — and both clearly beat image-only, by roughly two F1 points.

Slide 11 — Results: visualised · Sevil · (~38s)

Sevil: Here's the same story as a picture — and notice the axis is zoomed in to the range 0.95 to 1.0, so the differences are actually visible.

On both bars, accuracy and macro-F1, text-only and fusion sit right at the top and are basically indistinguishable. Image-only is the one that visibly trails on both. So at the overall level, fusion *matches* the strongest single modality rather than beating it. But the average isn't the whole story — let me show you where fusion genuinely earns its place.

Slide 12 — Where fusion actually helps · Sevil · (~60s)

Sevil: This is the most important slide for understanding fusion. The overall average hides the per-class picture, so let's look at the two classes the image branch was *worst* at.

The first is *Sandal*. On the image-only model it scored an F1 of just 0.897 — it kept confusing sandals with shoes, which makes sense at this resolution. The second is *Innerwear*, at 0.941.

Now watch what fusion does. By leaning on the text whenever the picture is unsure, fusion lifts *Sandal* from 0.897 up to 0.968, and *Innerwear* from 0.941 up to 0.983. In other words, fusion **repairs the image branch's worst confusions**.

Because the text model is already near-perfect everywhere else, fusion's contribution doesn't show up in the headline average — it shows up right here, class by class, exactly where one modality was failing. And that is precisely what the theory predicted: let the modality that knows the answer take over, example by example.

Slide 13 — Limitations & honest scope · Sevil · (~50s)

Sevil: We also want to be honest about the limits of this study.

First, the images in the small dataset are only about 60 by 80 pixels, upscaled to 224, so the visual branch has a real ceiling. Second, the product names are *very* descriptive, so text alone nearly solves the task — which leaves little headroom for fusion to win outright. That's a property of this dataset, not a flaw in fusion. Third, we used a single held-out split rather than k-fold, a deliberate compute trade-off, so some variance remains. And fourth, our backbone is frozen, so the CNN can't fully specialise to fashion textures — unfreezing and fine-tuning it would be the natural next step.

Slide 14 — Takeaways · Sevil · (~54s)

Sevil: So, to wrap up. We built the full pipeline the project asked for — a CNN, an RNN, and a fusion model — end to end. We derived the mathematics behind every component, from convolution and pooling to the GRU gates, cross-entropy, Adam, and the fusion layer. The comparison is fair, because all three models share the same pipeline and encoders with no data leakage, and it's reproducible, with a fixed seed, a saved vocabulary, and a one-click Colab notebook.

And the central result: on this dataset the text nearly saturates the task, so fusion *ties* the best single modality and both *beat* the image model — while fusion's real, constructive benefit shows up per class, recovering the image branch's hardest categories like Sandal and Innerwear.

Thank you all very much for listening. We'd be happy to take any questions.

Q&A Prep — likely questions & one-line answers

Not part of the timed talk. Whoever owns that part of the project takes the question; keep answers short and confident.

Theory & maths

- **Q: Why a GRU instead of an LSTM — and what are the LSTM's gates?** — An LSTM keeps a separate cell state with three gates: an **input** gate (what new information to write), a **forget** gate (what to erase), and an **output** gate (what to expose as the hidden state); a GRU merges these into just an **update** and a **reset** gate with no separate cell, so it's lighter and faster — plenty for our short product names.
- **Q: How does the GRU avoid the vanishing-gradient problem?** — When the update gate is near zero the hidden state is copied almost unchanged, giving an additive "carry" path so gradients survive across many steps instead of shrinking.
- **Q: Why does a fully-connected layer over the concatenated features capture image–text correlations?** — Each hidden unit weighs evidence from both modalities, so after the ReLU it can fire only when a visual *and* a textual feature co-occur — a cross-modal conjunction that two separate classifiers can't represent.
- **Q: What is the gradient of softmax + cross-entropy, and why does it matter?** — It reduces to "predicted minus true" ($\hat{y} - y$) at the logits — bounded and non-saturating, which keeps training stable.
- **Q: Why Adam rather than plain SGD?** — Adam adapts a per-parameter step from the first and second gradient moments, handling the very different gradient scales of a frozen CNN, an embedding and a GRU without hand-tuning each layer.
- **Q: Convolution gives equivariance but pooling gives invariance — what's the difference?** — Shifting the input shifts the convolution's feature map (equivariance); pooling then summarises a region so small shifts no longer change the output (invariance).
- **Q: Why global average pooling instead of flatten-then-dense?** — GAP adds no parameters, resists overfitting, and is shift-robust, collapsing the feature map into a compact fixed-length vector.

Results & methodology

- **Q: Fusion didn't beat text-only — didn't the project fail?** — No: the goal was to test *whether* combining helps, and here text already saturates the task, so the fair result is that fusion ties the best modality, beats the weak one, and repairs the image branch's worst classes.
- **Q: You ran a single seed — how can you call the −0.004 gap "noise"?** — We can't fully quantify variance from one run; we infer it from the hyperparameter grid, where validation accuracy moved only within 0.995–0.996, and confirming it with multiple seeds or cross-validation is the honest next step.
- **Q: Why not use k-fold cross-validation?** — k-fold costs k× the compute for a variance estimate we didn't strictly need; a fixed stratified 70/15/15 split — validation to choose, test once — is the standard, compute-appropriate choice, documented as a deliberate trade-off.
- **Q: Why report macro-F1 instead of accuracy?** — The ten classes are imbalanced, and macro-F1 weights every class equally, so strong performance on big classes can't mask weak minority classes.
- **Q: Can fusion ever *hurt* performance?** — Yes — on already-easy classes the noisier image features can slightly dilute a near-perfect text signal (Fragrance dropped from 1.000 to 0.982), but the per-class repairs elsewhere offset it to an overall tie.
- **Q: How did you prevent data leakage between splits?** — The text vectorizer is adapted on the training split only and then persisted, and the split is fixed by seed, so all three models share identical data with nothing leaking from validation or test.

Design choices & data

- **Q: Why freeze the backbone instead of fine-tuning it?** — Freezing prevents overfitting the ~5-million-parameter backbone on a small dataset, cuts training cost, and keeps BatchNorm stable; unfreezing the top blocks is our documented stretch goal and would most help the weak image branch.
- **Q: Why EfficientNet-B0 rather than ResNet or VGG?** — B0 uses compound scaling for the best accuracy-per-FLOP and is the smallest variant (~5 million parameters), which comfortably fits the free Colab GPU.
- **Q: Your images are 60×80 upscaled to 224 — isn't that a problem?** — Yes, it caps the visual branch's ceiling; we accept it as a course-project trade-off, and it's precisely why the image side has limited room to contribute.
- **Q: On what kind of task would fusion clearly win?** — One where neither modality is individually decisive — harder labels like `articleType`, noisier or shorter text, or lower-quality images — there fusion would have real headroom to beat both.
- **Q: Why trainable embeddings instead of GloVe or BERT?** — The descriptions are short and domain-specific, so a compact embedding learned end-to-end is lighter and sufficient; BERT would be overkill and far heavier to train.

Rapid-fire project Q&A (full-sentence answers)

Simple, factual questions an examiner might fire off quickly, answered in complete sentences.

Data & problem

- **Q: Which dataset did you use?** — We used the Kaggle *Fashion Product Images (small)* dataset, which conveniently ships each product's image, its category labels, and a short text field together in one place.
- **Q: What exactly does the model predict?** — It predicts the product's `subCategory`, and we restricted the problem to the ten most frequent sub-categories so the task is balanced enough to compare the three models fairly.
- **Q: What is the text input?** — The text modality is the `productDisplayName` field, which is the short product title such as "Nike Men Black Casual Shoes".
- **Q: Why did you choose the top-10 `subCategory` as the label?** — `masterCategory` is almost perfectly solvable from the image alone so fusion would add nothing visible, and `articleType` has over 140 classes which is too hard to train fairly, so the top-10 `subCategory` is the sweet spot where the second modality can make a measurable difference.
- **Q: How did you split the data?** — We used a stratified 70/15/15 train/validation/test split with a fixed seed, and all three models share exactly the same split so any difference comes from the model rather than the data.
- **Q: How many test samples are there?** — The held-out test set contains 3,000 products that the models never see during training.
- **Q: What image size do you feed the network?** — The images are resized to 224×224 pixels, upscaled from the dataset's small roughly 60×80 originals.

Image branch (CNN)

- **Q: Which CNN do you use, and is it trained?** — We use EfficientNetB0 with ImageNet weights as the backbone, and we keep it frozen so that only the small projection head on top is actually trained.
- **Q: Why do you freeze the backbone?** — Freezing reuses strong general-purpose ImageNet features, which prevents overfitting on our modest dataset, makes training much cheaper, and keeps the BatchNorm layers stable in inference mode.
- **Q: Why EfficientNetB0 specifically?** — It gives the best accuracy-per-computation thanks to compound scaling and is the smallest variant at around five million parameters, so it runs comfortably on a free Colab GPU.
- **Q: What does the pooling step do?** — We use global average pooling, which averages each feature channel down to a single number and gives us a compact, fixed-length image vector with no extra parameters.

Text branch (RNN)

- **Q: Which RNN do you use, and how big is it?** — We use a GRU with 128 units, which reads the description one word at a time while keeping a running memory of what it has seen.
- **Q: Why a GRU instead of an LSTM?** — A GRU has three gates instead of the LSTM's four, so it is lighter and faster to train, and that extra LSTM capacity is not needed for our short product names.
- **Q: What kind of word embeddings did you use?** — We learn a trainable 128-dimensional embedding from scratch alongside the model, rather than loading pretrained GloVe or BERT vectors.

- **Q: What are your vocabulary and sequence-length limits?** — We cap the vocabulary at 10,000 words and pad or truncate every description to a fixed length of 20 tokens.

Fusion

- **Q: How does your fusion actually work?** — We concatenate the 256-number image vector and the 128-number text vector into a single 384-number vector, then pass it through a dense layer with ReLU, a dropout of 0.3, and finally a softmax over the ten classes.
- **Q: Is this early or late fusion, and why did you choose it?** — It is early, feature-level fusion because we combine the two learned feature vectors before the decision, and we chose it so the joint layer can learn cross-modal interactions that decision-level (late) fusion simply cannot capture.
- **Q: Why does the fully-connected layer matter?** — Each neuron in that layer sees both modalities at once, so it can fire only when a visual feature and a textual feature occur together, which is exactly how the model learns image–text correlations and lets the text break ties when the image is ambiguous.
- **Q: How many neurons are in the head?** — The fusion hidden layer has 256 neurons and the output layer has 10 (one per class), while the two encoders feeding it produce 256 image features and 128 text features.

Training & loss

- **Q: What loss function do you use?** — We use sparse categorical cross-entropy, which penalises the model whenever it assigns a low probability to the correct class.
- **Q: Which optimizer did you use, and why that one?** — We use Adam with a learning rate of 0.001, because its per-parameter adaptive step sizes handle the very different gradient scales of a frozen CNN, an embedding, and a GRU without any manual per-layer tuning.
- **Q: How do you regularize the model?** — We rely on a dropout of 0.3 on the fusion layer, early stopping on the validation loss, the frozen backbone as an implicit regulariser, and a vocabulary built only from the training split.
- **Q: What batch size and how many epochs?** — We train with a batch size of 32 for up to 15 epochs, stopping early with a patience of 3 and restoring the best weights.

Evaluation & results

- **Q: Which metrics do you report, and why macro-F1?** — We report accuracy and macro-F1, and we emphasise macro-F1 because the ten classes are imbalanced and it weights every class equally instead of letting the large classes dominate the score.
- **Q: What were your results?** — On the held-out test set the image-only model reached a macro-F1 of 0.9727, the text-only model 0.9949, and the fusion model 0.9905.
- **Q: Did fusion beat the single-modality models?** — Fusion effectively ties the text-only model, with the gap well inside run-to-run noise, and it clearly beats the image-only model, because the very descriptive product names let text nearly saturate the task on its own.
- **Q: So where does fusion actually help?** — Its benefit shows up at the per-class level, where it repairs the image branch’s worst confusions — for example Sandal improves from 0.897 to 0.968 and Innerwear from 0.941 to 0.983.

- **Q: Why didn’t you use k-fold cross-validation?** — k-fold would cost several times the compute for a variance estimate we did not strictly need, so we used a single stratified held-out split as a deliberate and documented trade-off.
- **Q: How did you prevent data leakage?** — The text vectorizer is adapted only on the training split and then persisted, and the split itself is fixed by the seed, so nothing from validation or test ever leaks into training.